

ENEL445 — Test Revision Notes

Weeks 1–6: Gradient Methods · LS/Ridge/LASSO · Constrained Optimisation · MPC · Nelder–Mead

Notation Convention

$x_1, x_2, \dots, x_n$	components of the design variable vector (lowercase = dimension index)
$X_0, X_1, X_2, \dots, X_k$	sequence of iterates (uppercase = iteration number)
$\hat{X}_k$	candidate point while α is still unknown
p_k	search direction at iteration k
α_k	step size at iteration k , found by line search
$\phi(\alpha)$	1D slice of f along the search line (line search function only)
$\nabla f(X_k)$	gradient of f evaluated at iterate X_k
$H_f(X_k)$	Hessian of f evaluated at iterate X_k

Key distinction: “objective function” = $f(X)$. $\phi(\alpha) = f(\hat{X}_k)$ is the *line search function* — a 1D slice of f along the search direction. Once α is solved, $\hat{X}_k$ becomes the confirmed iterate X_k .

General Gradient-Based Update Rule

Every gradient-based method iterates:

$$X_k = X_{k-1} + \alpha_{k-1} p_{k-1}$$

Methods differ *only* in how p_{k-1} is chosen.

Optimality Conditions (Unconstrained)

Condition	Mathematical statement	Meaning
1st-order necessary	$\nabla f(X^*) = 0$	f is flat at X^* (stationary point)
2nd-order necessary	$H_f(X^*)$ positive semi-definite	no downward curvature
Sufficient (minimum)	$\nabla f(X^*) = 0$ and $H_f(X^*)$ positive definite	confirmed strong minimum

$\nabla f(X^*) = 0$ alone is not enough — saddle points and maxima also satisfy it. Check the Hessian to confirm a minimum.

Hessian sign summary: positive definite → minimum; negative definite → maximum; indefinite (mixed eigenvalue signs) → saddle point.

Algorithm 1 — Steepest Descent

Search direction: $p_{k-1} = -\nabla f(X_{k-1})$

Given $\min f(X)$ and starting point X_0 . Repeat for $k = 1, 2, \dots$

1. Compute $\nabla f(X_{k-1})$ *numeric vector*
2. Set $p_{k-1} = -\nabla f(X_{k-1})$ *search direction*
3. $\hat{X}_k = X_{k-1} + \alpha p_{k-1}$
e.g. $\hat{X}_1 = X_0 + \alpha p_0$ *vector in terms of α*
4. Substitute into f : $\phi(\alpha) = f(\hat{X}_k)$
5. Solve $\frac{d\phi}{d\alpha} = 0$ for α_{k-1} *exact line search*

6. Update: $X_k = X_{k-1} + \alpha_{k-1} p_{k-1}$

confirmed numeric point

Weakness: each search direction is perpendicular to the previous one (exact line search forces $\nabla f(X_k)^T p_{k-1} = 0$). On ill-conditioned functions (elongated contours), this causes *zigzagging* — the algorithm overshoots across the valley on each step. The condition number of H_f determines severity: larger ratio of max/min eigenvalue → worse zigzag.

Algorithm 2 — Conjugate Gradient (Fletcher–Reeves)

Search direction: $p_{k-1} = -\nabla f(X_{k-1}) + \beta_{k-1} p_{k-2}$ (for $k \geq 2$)

Given $\min f(X)$ and starting point X_0 .

Iteration 1 (identical to steepest descent, no previous direction):

1. Compute $\nabla f(X_0)$
2. Set $\beta_0 = 0$, $p_0 = -\nabla f(X_0)$
3. Form $\hat{X}_1 = X_0 + \alpha p_0$; compute $\phi(\alpha) = f(\hat{X}_1)$
4. Solve $\frac{d\phi}{d\alpha} = 0$ for α_0 ; update $X_1 = X_0 + \alpha_0 p_0$

Iteration $k \geq 2$ (conjugate correction applied):

1. Compute $\nabla f(X_{k-1})$
2. Compute β_{k-1} using **Fletcher–Reeves**:

$$\beta_{k-1} = \frac{\nabla f(X_{k-1})^T \nabla f(X_{k-1})}{\nabla f(X_{k-2})^T \nabla f(X_{k-2})}$$

3. Set $p_{k-1} = -\nabla f(X_{k-1}) + \beta_{k-1} p_{k-2}$ *corrected direction*
4. Form $\hat{X}_k = X_{k-1} + \alpha p_{k-1}$; compute $\phi(\alpha) = f(\hat{X}_k)$
5. Solve $\frac{d\phi}{d\alpha} = 0$ for α_{k-1} ; update $X_k = X_{k-1} + \alpha_{k-1} p_{k-1}$

Key property: for an n -dimensional quadratic objective, conjugate gradient converges in *exactly* n iterations. Steepest descent has no such guarantee.

Polak–Ribière alternative for β_{k-1} :

$$\beta_{k-1} = \frac{\nabla f(X_{k-1})^T (\nabla f(X_{k-1}) - \nabla f(X_{k-2}))}{\nabla f(X_{k-2})^T \nabla f(X_{k-2})}$$

If $\beta_{k-1} < 0$, set $\beta_{k-1} = 0$ (resets to steepest descent for that step). Polak–Ribière converges faster empirically on nonlinear problems.

Algorithm 3 — Newton’s Method

Search direction: $p_{k-1} = -H_f(X_{k-1})^{-1} \nabla f(X_{k-1})$

Given $\min f(X)$ and starting point X_0 . Repeat for $k = 1, 2, \dots$

1. Compute $\nabla f(X_{k-1})$ *numeric vector*
2. Compute $H_f(X_{k-1})$ *Hessian matrix*
3. Invert: $H_f(X_{k-1})^{-1}$ *for 2×2 : swap diagonal, negate off-diagonal, divide by det*
4. Set $p_{k-1} = -H_f(X_{k-1})^{-1} \nabla f(X_{k-1})$ *Newton direction*

5. Try $\alpha = 1$: form $\hat{X}_k = X_{k-1} + p_{k-1}$ *pure Newton step*
6. If needed, line search: $\phi(\alpha) = f(\hat{X}_k)$; solve $d\phi/d\alpha = 0$ for α_{k-1}
7. Update: $X_k = X_{k-1} + \alpha_{k-1} p_{k-1}$

2×2 inverse: $\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$

Key property: quadratic convergence near minimum — converges in one step for a quadratic f . High cost per iteration (Hessian computation and inversion), but very few iterations needed.

Algorithm 4 — BFGS (Quasi-Newton)

Search direction: $p_{k-1} = -\tilde{V}_{k-1} \nabla f(X_{k-1})$ where $\tilde{V}_{k-1} \approx H_f(X_{k-1})^{-1}$ is built iteratively

Initialise: $\tilde{V}_0 = I$ (identity). Repeat for $k = 1, 2, \dots$

1. Compute $\nabla f(X_{k-1})$
2. Set $p_{k-1} = -\tilde{V}_{k-1} \nabla f(X_{k-1})$ *quasi-Newton direction*
3. Form $\hat{X}_k = X_{k-1} + \alpha p_{k-1}$; compute $\phi(\alpha) = f(\hat{X}_k)$
4. Solve $d\phi/d\alpha = 0$ for α_{k-1} ; update $X_k = X_{k-1} + \alpha_{k-1} p_{k-1}$
5. Compute correction vectors:

$$s_{k-1} = X_k - X_{k-1}, \quad y_{k-1} = \nabla f(X_k) - \nabla f(X_{k-1})$$

6. Compute scalar: $\sigma_{k-1} = \frac{1}{y_{k-1}^T s_{k-1}}$

7. Update inverse Hessian approximation:

$$\tilde{V}_k = (I - \sigma_{k-1} s_{k-1} y_{k-1}^T) \tilde{V}_{k-1} (I - \sigma_{k-1} y_{k-1} s_{k-1}^T) + \sigma_{k-1} s_{k-1} s_{k-1}^T$$

Key property: no Hessian computation required — $\tilde{V}_k$ is built from gradient differences only. First iteration ($\tilde{V}_0 = I$) is identical to steepest descent.

Memory: stores one $n \times n$ matrix $\tilde{V}_k$ (or $\mathcal{O}(n^2)$ entries). L-BFGS variant stores only the last m correction vectors ($\mathcal{O}(mn)$) for large-scale problems.

Comparison

Method	Direction p_{k-1}	Memory	Weakness
Steepest descent	$-\nabla f(X_{k-1})$	None	Zigzags on ill-conditioned problems
Conjugate gradient	$-\nabla f(X_{k-1}) + \beta_{k-1} p_{k-2}$	One step back	Resets needed for nonlinear f
Newton	$-H_f(X_{k-1})^{-1} \nabla f(X_{k-1})$	Full curvature	Requires Hessian computation
BFGS	$-\tilde{V}_{k-1} \nabla f(X_{k-1})$	Approx. inverse Hessian $\tilde{V}_{k-1}$	No Hessian needed; $\tilde{V}_0 = I$

Mathematical Background

Concept	Formula
Gradient	$\nabla f(X) = [\partial f / \partial x_1, \dots, \partial f / \partial x_n]^T$
Hessian	$[H_f]_{ij} = \partial^2 f / \partial x_i \partial x_j$; symmetric
2nd-order Taylor	$f(X+p) \approx f(X) + \nabla f(X)^T p + \frac{1}{2} p^T H_f(X) p$
Convexity	f convex $\Leftrightarrow H_f \succeq 0$ everywhere ; local min = global min
Numerical gradient	$\partial f / \partial x_i \approx [f(X + h e_i) - f(X)] / h$; $h \approx 10^{-6}$

Least Squares & Regularisation

Problem: fit $A\theta \approx b$ where $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$.

Method	Objective	Solution
Least squares	$\ A\theta - b\ _2^2$	$\theta^* = (A^T A)^{-1} A^T b$ (normal equations)
Ridge (L^2)	$\ A\theta - b\ _2^2 + \lambda \ \theta\ _2^2$	$\theta^* = (A^T A + \lambda I)^{-1} A^T b$
LASSO (L^1)	$\ A\theta - b\ _2^2 + \lambda \ \theta\ _1$	no closed form; induces sparsity

Ridge: $A^T A + \lambda I$ always invertible; shrinks all coefficients uniformly. LASSO soft-threshold: $\text{prox}_\lambda(z) = \text{sign}(z) \max(|z| - \lambda, 0)$.

Majorization–Minimization (MM): at iterate θ^k , build surrogate $g(\theta, \theta^k) \geq f(\theta)$ with $g(\theta^k, \theta^k) = f(\theta^k)$; minimise g ; repeat. Guarantees $f(\theta^{k+1}) \leq f(\theta^k)$. MM generalises EM and proximal gradient.

Constrained Optimisation

Standard form: $\min_X f(X)$ s.t. $h_j(X) = 0$ ($j = 1, \dots, m$), $g_i(X) \leq 0$ ($i = 1, \dots, p$)

Lagrangian:

$$\mathcal{L}(X, \lambda, \mu) = f(X) + \sum_j \lambda_j h_j(X) + \sum_i \mu_i g_i(X)$$

KKT Conditions (necessary at local min X^* under regularity; sufficient for convex problems):

1. **Stationarity:** $\nabla_X \mathcal{L}(X^*, \lambda^*, \mu^*) = 0$
2. **Primal feasibility:** $h_j(X^*) = 0 \forall j$; $g_i(X^*) \leq 0 \forall i$
3. **Dual feasibility:** $\mu_i^* \geq 0 \forall i$
4. **Complementary slackness:** $\mu_i^* g_i(X^*) = 0 \forall i$ (inactive constraint $\Rightarrow \mu_i^* = 0$)

Penalty method: add $\frac{c}{2} \sum_i \max(0, g_i)^2$; solve unconstrained; increase c

Interior point: barrier $-\mu \sum_i \log(-g_i)$; stays strictly feasible; decrease $\mu \rightarrow 0$

SQP: solve quadratic approximation to constrained problem at each iterate

Duality: $q(\lambda, \mu) = \inf_X \mathcal{L}(X, \lambda, \mu)$; dual problem $\max_{\lambda, \mu \geq 0} q$. Weak duality: $q \leq p^*$ always. Strong duality (zero gap): convex f, g , affine h , Slater's condition ($\exists$ strictly feasible X).

Optimal Control & MPC

Finite-horizon problem (given x_0):

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} c(x_k, u_k) + V_f(x_N) \quad \text{s.t.} \quad x_{k+1} = f(x_k, u_k), \quad x_k \in \mathcal{X}, \quad u_k \in \mathcal{U}$$

Dynamic programming (backward induction):

$$V_N(x) = V_f(x); \quad V_k(x) = \min_{u \in \mathcal{U}(x)} [c(x, u) + V_{k+1}(f(x, u))]$$

MPC feedback law: $\mu_k(x) = \arg \min_u [c(x, u) + V_{k+1}(f(x, u))]$. Bellman's principle: sub-sequences of an optimal sequence are optimal.

Receding horizon (MPC): solve finite-horizon problem from current x_k ; apply u_0^* only; advance; repeat.

Advantages handles hard constraints; follows optimal trajectory; works well with short horizons

Disadvantages requires accurate model; computationally expensive; stability hard to prove

Nelder–Mead (Gradient-Free)

Simplex: $n+1$ vertices in $\mathbb{R}^n$. Order as $X^{(0)}, \dots, X^{(n)}$ with $f(X^{(0)}) \leq \dots \leq f(X^{(n)})$ (best to worst).

Centroid (exclude worst): $\bar{X} = \frac{1}{n} \sum_{i=0}^{n-1} X^{(i)}$

Standard parameters: $\alpha = 1$ (reflect), $\gamma = 2$ (expand), $\rho = \frac{1}{2}$ (contract), $\sigma = \frac{1}{2}$ (shrink).

Each iteration: order, compute $\bar{X}$, reflect: $X_r = \bar{X} + \alpha(\bar{X} - X^{(n)})$

1. $f(X^{(0)}) \leq f(X_r) < f(X^{(n-1)})$: **replace** $X^{(n)} \leftarrow X_r$. (*accept reflection*)
2. $f(X_r) < f(X^{(0)})$: **expand** $X_e = \bar{X} + \gamma(X_r - \bar{X})$; **replace** $X^{(n)} \leftarrow \arg \min(X_r, X_e)$.
3. $f(X^{(n-1)}) \leq f(X_r) < f(X^{(n)})$: **outer contract** $X_{oc} = \bar{X} + \rho(X_r - \bar{X})$; if $f(X_{oc}) \leq f(X_r)$: $X^{(n)} \leftarrow X_{oc}$; **else shrink**.
4. $f(X_r) \geq f(X^{(n)})$: **inner contract** $X_{ic} = \bar{X} - \rho(\bar{X} - X^{(n)})$; if $f(X_{ic}) < f(X^{(n)})$: $X^{(n)} \leftarrow X_{ic}$; **else shrink**.
5. **Shrink** (last resort): $X^{(i)} \leftarrow X^{(0)} + \sigma(X^{(i)} - X^{(0)})$ for $i = 1, \dots, n$.

Nelder–Mead requires only function evaluations (zeroth-order / derivative-free). Useful for noisy, discontinuous, or black-box f . Local search only — no global optimality guarantee. KKT conditions do not apply (no smoothness required).